Azure.AI.TextAnalytics 5.3.0

Prefix Reserved (https://docs.microsoft.com/nuget/nuget-org/id-prefix-reservation)

.NET Standard 2.0 (/packages/Azure.AI.TextAnalytics#supportedframeworks-body-tab)

.NET CLI | PMC | PackageReference | CPM | Paket CLI | Script & Interactive | File-based Apps | Cake

```
> dotnet add package Azure.AI.TextAnalytics --version 5.3.0
```

README | Frameworks | Dependencies | Used By | Versions | Release Notes

Azure Cognitive Services Text Analytics client library for .NET

Text Analytics is part of the Azure Cognitive Service for Language, a cloud-based service that provides Natural Language Processing (NLP) features for under

- Language detection
- Sentiment analysis
- Key phrase extraction
- Named entity recognition (NER)
- Personally identifiable information (PII) entity recognition
- Entity linking
- Text analytics for health
- Custom named entity recognition (Custom NER)
- Custom text classification
- Extractive text summarization
- Abstractive text summarization

Source code (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/src) | Package (NuGet) (https://www.nuge ref-docs) | Product documentation (https://docs.microsoft.com/azure/cognitive-services/language-service/) | Samples (https://github.com/Azure/azure

Getting started

Install the package

Install the Azure Text Analytics client library for .NET with **NuGet (https://www.nuget.org/)**:

```
dotnet add package Azure.AI.TextAnalytics
```

This table shows the relationship between SDK versions and supported API versions of the service:

> Note that `5.2.0` is the first stable version of the client library that targets the Azure Cognitive Service for Language APIs which includes the existing text service API has changed from semantic to date-based versioning.

SDK version	Supported API version of service
5.3.X	3.0, 3.1, 2022-05-01, 2023-04-01 (default)
5.2.X	3.0, 3.1, 2022-05-01 (default)
5.1.X	3.0, 3.1 (default)
5.0.X	3.0

SDK version	Supported API version of service
1.0.X	3.0

Prerequisites

- An **Azure subscription (https://azure.microsoft.com/free/dotnet/)**.
- An existing Cognitive Services or Language service resource.

Create a Cognitive Services resource or a Language service resource

Azure Cognitive Service for Language supports both **multi-service and single-service access (https://learn.microsoft.com/azure/cognitive-services/cognit** services under a single endpoint and API key. To access the features of the Language service only, create a Language service resource instead.

You can create either resource via the **Azure portal (https://learn.microsoft.com/azure/cognitive-services/cognitive-services-apis-create-account)** or, alte **services/cognitive-services-apis-create-account-cli)** to create it using the **Azure CLI (https://docs.microsoft.com/cli/azure)**.

Authenticate the client

Interaction with the service using the client library begins with creating an instance of the **TextAnalyticsClient (https://github.com/Azure/azure-sdk-for-net** and either an **API key** or `TokenCredential` to instantiate a client object. For more information regarding authenticating with cognitive services, see **Authen services/authentication)**.

Get an API key

You can get the `endpoint` and `API key` from the Cognitive Services resource or Language service resource information in the **Azure Portal (https://portal.a**

Alternatively, use the **Azure CLI (https://docs.microsoft.com/cli/azure)** snippet below to get the API key from the Language service resource.

```
az cognitiveservices account keys list --resource-group <your-resource-group-name> --name <your-resource-name>
```

Create a `TextAnalyticsClient` using an API key credential

Once you have the value for the API key, create an `AzureKeyCredential`. This will allow you to update the API key without creating a new client.

With the value of the endpoint and an `AzureKeyCredential`, you can create the **TextAnalyticsClient (https://github.com/Azure/azure-sdk-for-net/tree/ma**

```
Uri endpoint = new("<endpoint>");
AzureKeyCredential credential = new("<apiKey>");
TextAnalyticsClient client = new(endpoint, credential);
```

Create a `TextAnalyticsClient` with an Azure Active Directory credential

Client API key authentication is used in most of the examples in this getting started guide, but you can also authenticate with Azure Active Directory using th Note that regional endpoints do not support AAD authentication. Create a **custom subdomain (https://docs.microsoft.com/azure/cognitive-services/auth** authentication.

To use the **DefaultAzureCredential (https://github.com/Azure/azure-sdk-for-net/blob/main/sdk/identity/Azure.Identity/README.md#defaultazurecrede** Azure.Identity package:

```
dotnet add package Azure.Identity
```

You will also need to **register a new AAD application (https://docs.microsoft.com/azure/cognitive-services/authentication#assign-a-role-to-a-service-pri a-service-principal)** to the Language service by assigning the `"Cognitive Services User"` role to your service principal.

Set the values of the client ID, tenant ID, and client secret of the AAD application as environment variables: AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_C

```
Uri endpoint = new("<endpoint>");
TextAnalyticsClient client = new(endpoint, new DefaultAzureCredential());
```

Key concepts

`TextAnalyticsClient`

A `TextAnalyticsClient` is the primary interface for developers using the Text Analytics client library. It provides both synchronous and asynchronous opera

Input

A **document**, is a single unit of input to be analyzed by the predictive models in the Language service. Operations on `TextAnalyticsClient` may take a sing size, and supported text encoding see **here (https://aka.ms/azsdk/textanalytics/data-limits)**.

Operation on multiple documents

For each supported operation, `TextAnalyticsClient` provides a method that accepts a batch of documents as strings, or a batch of either `TextDocumentInp` the documents in the batch are written in different languages, or provide a country hint about the language of the document.

Note: It is recommended to use the batch methods when working on production environments as they allow you to send one request with multiple docume

Return value

Return values, such as `AnalyzeSentimentResult`, is the result of a Text Analytics operation, containing a prediction or predictions about a single document. A processed.

Return value Collection

A Return value collection, such as `AnalyzeSentimentResultCollection`, is a collection of operation results, where each corresponds to one of the document collections. The return value also contains a `HasError` property that allows to identify if an operation executed was successful or unsuccessful for the given

Long-Running Operations

For large documents which take a long time to execute, these operations are implemented as **long-running operations (https://github.com/Azure/azure-s operationt)**. Long-running operations consist of an initial request sent to the service to start an operation, followed by polling the service at intervals to dete

For long running operations in the Azure SDK, the client exposes a `Start<operation-name>` method that returns an `Operation<T>` or a `PageableOperation` obtain its result. A sample code snippet is provided to illustrate using long-running operations **below**.

Thread safety

We guarantee that all client instance methods are thread-safe and independent of each other (**guideline (https://azure.github.io/azure-sdk/dotnet_introdu** instances is always safe, even across threads.

Additional concepts

Client options (https://github.com/Azure/azure-sdk-for-net/blob/main/sdk/core/Azure.Core/README.md#configuring-service-clients-using-clientoptic net/blob/main/sdk/core/Azure.Core/README.md#accessing-http-response-details-using-responset) | Handling failures (https://github.com/Azure/azur Diagnostics (https://github.com/Azure/azure-sdk-for-net/blob/main/sdk/core/Azure.Core/samples/Diagnostics.md) | Mocking (https://github.com/Azu (https://devblogs.microsoft.com/azure-sdk/lifetime-management-and-thread-safety-guarantees-of-azure-sdk-net-clients/)

Examples

The following section provides several code snippets using the `client` **created above**, and covers the main features present in this client library. Although r package supports both synchronous and asynchronous APIs.

Sync examples

- **Detect Language**
- **Analyze Sentiment**
- **Extract Key Phrases**
- **Recognize Named Entities**
- **Recognize PII Entities**
- **Recognize Linked Entities**

Async examples

- **Detect Language Asynchronously**
- **Recognize Named Entities Asynchronously**
- **Analyze Healthcare Entities Asynchronously**
- **Run multiple actions Asynchronously**

Detect Language

Run a predictive model to determine the language that the passed-in document or batch of documents are written in.

```
string document =
    "Este documento está escrito en un lenguaje diferente al inglés. Su objectivo es demostrar cómo"
    + " invocar el método de Detección de Lenguaje del servicio de Text Analytics en Microsoft Azure."
    + " También muestra cómo acceder a la información retornada por el servicio. Esta funcionalidad es"
    + " útil para los sistemas de contenido que recopilan texto arbitrario, donde el lenguaje no se conoce"
    + " de antemano. Puede usarse para detectar una amplia gama de lenguajes, variantes, dialectos y"
    + " algunos idiomas regionales o culturales.";

try
{
    Response<DetectedLanguage> response = client.DetectLanguage(document);
    DetectedLanguage language = response.Value;

    Console.WriteLine($"Detected language is {language.Name} with a confidence score of {language.ConfidenceScore}.");
}
catch (RequestFailedException exception)
{
    Console.WriteLine($"Error Code: {exception.ErrorCode}");
    Console.WriteLine($"Message: {exception.Message}");
}
```

For samples on using the production recommended option `DetectLanguageBatch` see **here (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/t**

Please refer to the service documentation for a conceptual discussion of **language detection (https://docs.microsoft.com/azure/cognitive-services/langua**

Analyze Sentiment

Run a predictive model to determine the positive, negative, neutral or mixed sentiment contained in the passed-in document or batch of documents.

```
string document =
    "I had the best day of my life. I decided to go sky-diving and it made me appreciate my whole life so"
    + "much more. I developed a deep-connection with my instructor as well, and I feel as if I've made a"
    + "life-long friend in her.";

try
{
    Response<DocumentSentiment> response = client.AnalyzeSentiment(document);
    DocumentSentiment docSentiment = response.Value;

    Console.WriteLine($"Document sentiment is {docSentiment.Sentiment} with: ");
    Console.WriteLine($"  Positive confidence score: {docSentiment.ConfidenceScores.Positive}");
    Console.WriteLine($"  Neutral confidence score: {docSentiment.ConfidenceScores.Neutral}");
    Console.WriteLine($"  Negative confidence score: {docSentiment.ConfidenceScores.Negative}");
}
catch (RequestFailedException exception)
{
    Console.WriteLine($"Error Code: {exception.ErrorCode}");
    Console.WriteLine($"Message: {exception.Message}");
}
```

For samples on using the production recommended option `AnalyzeSentimentBatch` see **here (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk**

To get more granular information about the opinions related to targets of a product/service, also known as Aspect-based Sentiment Analysis in Natural Lang
(https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample2.1_AnalyzeSentimentWithOpinionMir

Please refer to the service documentation for a conceptual discussion of **sentiment analysis (https://docs.microsoft.com/azure/cognitive-services/languag**

Extract Key Phrases

Run a model to identify a collection of significant phrases found in the passed-in document or batch of documents.

```
string document =
    "My cat might need to see a veterinarian. It has been sneezing more than normal, and although my"
    + " little sister thinks it is funny, I am worried it has the cold that I got last week. We are going"
    + " to call tomorrow and try to schedule an appointment for this week. Hopefully it will be covered by"
    + " the cat's insurance. It might be good to not let it sleep in my room for a while.";

try
{
    Response<KeyPhraseCollection> response = client.ExtractKeyPhrases(document);
    KeyPhraseCollection keyPhrases = response.Value;

    Console.WriteLine($"Extracted {keyPhrases.Count} key phrases:");
    foreach (string keyPhrase in keyPhrases)
    {
        Console.WriteLine($"  {keyPhrase}");
    }
}
catch (RequestFailedException exception)
{
    Console.WriteLine($"Error Code: {exception.ErrorCode}");
    Console.WriteLine($"Message: {exception.Message}");
}
```

For samples on using the production recommended option `ExtractKeyPhrasesBatch` see **here (https://github.com/Azure/azure-sdk-for-net/tree/main/sd**

Please refer to the service documentation for a conceptual discussion of **key phrase extraction (https://docs.microsoft.com/azure/cognitive-services/langu**

Recognize Named Entities

Run a predictive model to identify a collection of named entities in the passed-in document or batch of documents and categorize those entities into categc
Named Entity Categories (https://docs.microsoft.com/azure/cognitive-services/language-service/named-entity-recognition/concepts/named-entity-cat

```
string document =
    "We love this trail and make the trip every year. The views are breathtaking and well worth the hike!"
    + " Yesterday was foggy though, so we missed the spectacular views. We tried again today and it was"
    + " amazing. Everyone in my family liked the trail although it was too challenging for the less"
    + " athletic among us. Not necessarily recommended for small children. A hotel close to the trail"
    + " offers services for childcare in case you want that.";

try
{
    Response<CategorizedEntityCollection> response = client.RecognizeEntities(document);
    CategorizedEntityCollection entitiesInDocument = response.Value;

    Console.WriteLine($"Recognized {entitiesInDocument.Count} entities:");
    foreach (CategorizedEntity entity in entitiesInDocument)
    {
        Console.WriteLine($"  Text: {entity.Text}");
        Console.WriteLine($"  Offset: {entity.Offset}");
        Console.WriteLine($"  Length: {entity.Length}");
        Console.WriteLine($"  Category: {entity.Category}");
        if (!string.IsNullOrEmpty(entity.SubCategory))
            Console.WriteLine($"  SubCategory: {entity.SubCategory}");
        Console.WriteLine($"  Confidence score: {entity.ConfidenceScore}");
        Console.WriteLine();
    }
}
catch (RequestFailedException exception)
{
    Console.WriteLine($"Error Code: {exception.ErrorCode}");
    Console.WriteLine($"Message: {exception.Message}");
}
```

For samples on using the production recommended option `RecognizeEntitiesBatch` see **here (https://github.com/Azure/azure-sdk-for-net/tree/main/sd**

Please refer to the service documentation for a conceptual discussion of **named entity recognition (https://docs.microsoft.com/azure/cognitive-services/l**

Recognize PII Entities

Run a predictive model to identify a collection of entities containing Personally Identifiable Information found in the passed-in document or batch of docum
credit card number.

```
string document =
    "Parker Doe has repaid all of their loans as of 2020-04-25. Their SSN is 859-98-0987. To contact them,"
    + " use their phone number 800-102-1100. They are originally from Brazil and have document ID number"
    + " 998.214.865-68.";

try
{
    Response<PiiEntityCollection> response = client.RecognizePiiEntities(document);
    PiiEntityCollection entities = response.Value;

    Console.WriteLine($"Redacted Text: {entities.RedactedText}");
    Console.WriteLine();
    Console.WriteLine($"Recognized {entities.Count} PII entities:");
    foreach (PiiEntity entity in entities)
    {
        Console.WriteLine($"  Text: {entity.Text}");
        Console.WriteLine($"  Category: {entity.Category}");
        if (!string.IsNullOrEmpty(entity.SubCategory))
            Console.WriteLine($"  SubCategory: {entity.SubCategory}");
        Console.WriteLine($"  Confidence score: {entity.ConfidenceScore}");
        Console.WriteLine();
    }
}
catch (RequestFailedException exception)
{
    Console.WriteLine($"Error Code: {exception.ErrorCode}");
    Console.WriteLine($"Message: {exception.Message}");
}
```

For samples on using the production recommended option `RecognizePiiEntitiesBatch` see **here (https://github.com/Azure/azure-sdk-for-net/tree/main**

Please refer to the service documentation for supported **PII entity types (https://docs.microsoft.com/azure/cognitive-services/language-service/personal**

Recognize Linked Entities

Run a predictive model to identify a collection of entities found in the passed-in document or batch of documents, and include information linking the entit

```
string document =
    "Microsoft was founded by Bill Gates with some friends he met at Harvard. One of his friends, Steve"
    + " Ballmer, eventually became CEO after Bill Gates as well. Steve Ballmer eventually stepped down as"
    + " CEO of Microsoft, and was succeeded by Satya Nadella. Microsoft originally moved its headquarters"
    + " to Bellevue, Washington in Januaray 1979, but is now headquartered in Redmond.";

try
{
    Response<LinkedEntityCollection> response = client.RecognizeLinkedEntities(document);
    LinkedEntityCollection linkedEntities = response.Value;

    Console.WriteLine($"Recognized {linkedEntities.Count} entities:");
    foreach (LinkedEntity linkedEntity in linkedEntities)
    {
        Console.WriteLine($"  Name: {linkedEntity.Name}");
        Console.WriteLine($"  Language: {linkedEntity.Language}");
        Console.WriteLine($"  Data Source: {linkedEntity.DataSource}");
        Console.WriteLine($"  URL: {linkedEntity.Url}");
        Console.WriteLine($"  Entity Id in Data Source: {linkedEntity.DataSourceEntityId}");
        foreach (LinkedEntityMatch match in linkedEntity.Matches)
        {
            Console.WriteLine($"    Match Text: {match.Text}");
            Console.WriteLine($"    Offset: {match.Offset}");
            Console.WriteLine($"    Length: {match.Length}");
            Console.WriteLine($"    Confidence score: {match.ConfidenceScore}");
        }
        Console.WriteLine();
    }
}
catch (RequestFailedException exception)
{
    Console.WriteLine($"Error Code: {exception.ErrorCode}");
    Console.WriteLine($"Message: {exception.Message}");
}
```

For samples on using the production recommended option `RecognizeLinkedEntitiesBatch` see **here (https://github.com/Azure/azure-sdk-for-net/tree/m**

Please refer to the service documentation for a conceptual discussion of **entity linking (https://docs.microsoft.com/azure/cognitive-services/language-ser**

Detect Language Asynchronously

Run a predictive model to determine the language that the passed-in document or batch of documents are written in.

```
string document =
    "Este documento está escrito en un lenguaje diferente al inglés. Su objectivo es demostrar cómo"
    + " invocar el método de Detección de Lenguaje del servicio de Text Analytics en Microsoft Azure."
    + " También muestra cómo acceder a la información retornada por el servicio. Esta funcionalidad es"
    + " útil para los sistemas de contenido que recopilan texto arbitrario, donde el lenguaje no se conoce"
    + " de antemano. Puede usarse para detectar una amplia gama de lenguajes, variantes, dialectos y"
    + " algunos idiomas regionales o culturales.";

try
{
    Response<DetectedLanguage> response = await client.DetectLanguageAsync(document);
    DetectedLanguage language = response.Value;

    Console.WriteLine($"Detected language is {language.Name} with a confidence score of {language.ConfidenceScore}.");
}
catch (RequestFailedException exception)
{
    Console.WriteLine($"Error Code: {exception.ErrorCode}");
    Console.WriteLine($"Message: {exception.Message}");
}
```

Recognize Named Entities Asynchronously

Run a predictive model to identify a collection of named entities in the passed-in document or batch of documents and categorize those entities into categc
Named Entity Categories (https://docs.microsoft.com/azure/cognitive-services/language-service/named-entity-recognition/concepts/named-entity-cat

```
string document =
    "We love this trail and make the trip every year. The views are breathtaking and well worth the hike!"
    + " Yesterday was foggy though, so we missed the spectacular views. We tried again today and it was"
    + " amazing. Everyone in my family liked the trail although it was too challenging for the less"
    + " athletic among us. Not necessarily recommended for small children. A hotel close to the trail"
    + " offers services for childcare in case you want that.";

try
{
    Response<CategorizedEntityCollection> response = await client.RecognizeEntitiesAsync(document);
    CategorizedEntityCollection entitiesInDocument = response.Value;

    Console.WriteLine($"Recognized {entitiesInDocument.Count} entities:");
    foreach (CategorizedEntity entity in entitiesInDocument)
    {
        Console.WriteLine($"  Text: {entity.Text}");
        Console.WriteLine($"  Offset: {entity.Offset}");
        Console.WriteLine($"  Length: {entity.Length}");
        Console.WriteLine($"  Category: {entity.Category}");
        if (!string.IsNullOrEmpty(entity.SubCategory))
            Console.WriteLine($"  SubCategory: {entity.SubCategory}");
        Console.WriteLine($"  Confidence score: {entity.ConfidenceScore}");
        Console.WriteLine();
    }
}
catch (RequestFailedException exception)
{
    Console.WriteLine($"Error Code: {exception.ErrorCode}");
    Console.WriteLine($"Message: {exception.Message}");
}
```

Analyze Healthcare Entities Asynchronously

Text Analytics for health is a containerized service that extracts and labels relevant medical information from unstructured texts such as doctor's notes, disch
Analytics for health (https://docs.microsoft.com/azure/cognitive-services/language-service/text-analytics-for-health/overview?tabs=ner).

```
string documentA =
    "RECORD #333582770390100 | MH | 85986313 | | 054351 | 2/14/2001 12:00:00 AM |"
    + " CORONARY ARTERY DISEASE | Signed | DIS |"
    + Environment.NewLine
    + " Admission Date: 5/22/2001 Report Status: Signed Discharge Date: 4/24/2001"
    + " ADMISSION DIAGNOSIS: CORONARY ARTERY DISEASE."
    + Environment.NewLine
    + " HISTORY OF PRESENT ILLNESS: The patient is a 54-year-old gentleman with a history of progressive"
    + " angina over the past several months. The patient had a cardiac catheterization in July of this"
    + " year revealing total occlusion of the RCA and 50% left main disease, with a strong family history"
    + " of coronary artery disease with a brother dying at the age of 52 from a myocardial infarction and"
    + " another brother who is status post coronary artery bypass grafting. The patient had a stress"
    + " echocardiogram done on July, 2001, which showed no wall motion abnormalities, but this was a"
    + " difficult study due to body habitus. The patient went for six minutes with minimal ST depressions"
    + " in the anterior lateral leads, thought due to fatigue and wrist pain, his anginal equivalent. Due"
    + " to the patient'sincreased symptoms and family history and history left main disease with total"
    + " occasional of his RCA was referred for revascularization with open heart surgery.";

string documentB = "Prescribed 100mg ibuprofen, taken twice daily.";

// Prepare the input of the text analysis operation. You can add multiple documents to this list and
// perform the same operation on all of them simultaneously.
List<string> batchedDocuments = new()
{
    documentA,
    documentB
};

// Perform the text analysis operation.
AnalyzeHealthcareEntitiesOperation operation = await client.AnalyzeHealthcareEntitiesAsync(WaitUntil.Completed, batchedDocuments);

Console.WriteLine($"The operation has completed.");
Console.WriteLine();

// View the operation status.
Console.WriteLine($"Created On   : {operation.CreatedOn}");
Console.WriteLine($"Expires On   : {operation.ExpiresOn}");
Console.WriteLine($"Id           : {operation.Id}");
Console.WriteLine($"Status       : {operation.Status}");
Console.WriteLine($"Last Modified: {operation.LastModified}");
Console.WriteLine();

// View the operation results.
await foreach (AnalyzeHealthcareEntitiesResultCollection documentsInPage in operation.Value)
{
    Console.WriteLine($"Analyze Healthcare Entities, model version: \"{documentsInPage.ModelVersion}\"");
    Console.WriteLine();

    foreach (AnalyzeHealthcareEntitiesResult documentResult in documentsInPage)
    {
        if (documentResult.HasError)
        {
            Console.WriteLine($"  Error!");
            Console.WriteLine($"  Document error code: {documentResult.Error.ErrorCode}");
            Console.WriteLine($"  Message: {documentResult.Error.Message}");
            continue;
        }

        Console.WriteLine($"  Recognized the following {documentResult.Entities.Count} healthcare entities:");
        Console.WriteLine();

        // View the healthcare entities that were recognized.
        foreach (HealthcareEntity entity in documentResult.Entities)
        {
            Console.WriteLine($"  Entity: {entity.Text}");
            Console.WriteLine($"  Category: {entity.Category}");
            Console.WriteLine($"  Offset: {entity.Offset}");
            Console.WriteLine($"  Length: {entity.Length}");
```

```
            Console.WriteLine($"  NormalizedText: {entity.NormalizedText}");
            Console.WriteLine($"  Links:");

            // View the entity data sources.
            foreach (EntityDataSource entityDataSource in entity.DataSources)
            {
                Console.WriteLine($"    Entity ID in Data Source: {entityDataSource.EntityId}");
                Console.WriteLine($"    DataSource: {entityDataSource.Name}");
            }

            // View the entity assertions.
            if (entity.Assertion is not null)
            {
                Console.WriteLine($"  Assertions:");

                if (entity.Assertion?.Association is not null)
                {
                    Console.WriteLine($"    Association: {entity.Assertion?.Association}");
                }

                if (entity.Assertion?.Certainty is not null)
                {
                    Console.WriteLine($"    Certainty: {entity.Assertion?.Certainty}");
                }

                if (entity.Assertion?.Conditionality is not null)
                {
                    Console.WriteLine($"    Conditionality: {entity.Assertion?.Conditionality}");
                }
            }
        }

        Console.WriteLine($"  We found {documentResult.EntityRelations.Count} relations in the current document:");
        Console.WriteLine();

        // View the healthcare entity relations that were recognized.
        foreach (HealthcareEntityRelation relation in documentResult.EntityRelations)
        {
            Console.WriteLine($"    Relation: {relation.RelationType}");
            if (relation.ConfidenceScore is not null)
            {
                Console.WriteLine($"    ConfidenceScore: {relation.ConfidenceScore}");
            }
            Console.WriteLine($"    For this relation there are {relation.Roles.Count} roles");

            // View the relation roles.
            foreach (HealthcareEntityRelationRole role in relation.Roles)
            {
                Console.WriteLine($"      Role Name: {role.Name}");

                Console.WriteLine($"      Associated Entity Text: {role.Entity.Text}");
                Console.WriteLine($"      Associated Entity Category: {role.Entity.Category}");
                Console.WriteLine();
            }

            Console.WriteLine();
        }

        Console.WriteLine();
    }
}
```

Run multiple actions Asynchronously

This functionality allows running multiple actions in one or more documents. Actions include:

- Named Entities Recognition
- PII Entities Recognition

- Linked Entity Recognition
- Key Phrase Extraction
- Sentiment Analysis
- Healthcare Entities Recognition (see sample **here (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/s**
- Custom Named Entities Recognition (see sample **here (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnaly**
- Custom Single Label Classification (see sample **here (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytic**
- Custom Multi Label Classification (see sample **here (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics**

```
string documentA =
    "We love this trail and make the trip every year. The views are breathtaking and well worth the hike!"
    + " Yesterday was foggy though, so we missed the spectacular views. We tried again today and it was"
    + " amazing. Everyone in my family liked the trail although it was too challenging for the less"
    + " athletic among us.";

string documentB =
    "Last week we stayed at Hotel Foo to celebrate our anniversary. The staff knew about our anniversary"
    + " so they helped me organize a little surprise for my partner. The room was clean and with the"
    + " decoration I requested. It was perfect!";

// Prepare the input of the text analysis operation. You can add multiple documents to this list and
// perform the same operation on all of them simultaneously.
List<string> batchedDocuments = new()
{
    documentA,
    documentB
};

TextAnalyticsActions actions = new()
{
    ExtractKeyPhrasesActions = new List<ExtractKeyPhrasesAction>() { new ExtractKeyPhrasesAction() { ActionName = "ExtractKeyPhr
    RecognizeEntitiesActions = new List<RecognizeEntitiesAction>() { new RecognizeEntitiesAction() { ActionName = "RecognizeEnti
    DisplayName = "AnalyzeOperationSample"
};

// Perform the text analysis operation.
AnalyzeActionsOperation operation = await client.AnalyzeActionsAsync(WaitUntil.Completed, batchedDocuments, actions);

// View the operation status.
Console.WriteLine($"Created On   : {operation.CreatedOn}");
Console.WriteLine($"Expires On   : {operation.ExpiresOn}");
Console.WriteLine($"Id           : {operation.Id}");
Console.WriteLine($"Status       : {operation.Status}");
Console.WriteLine($"Last Modified: {operation.LastModified}");
Console.WriteLine();

if (!string.IsNullOrEmpty(operation.DisplayName))
{
    Console.WriteLine($"Display name: {operation.DisplayName}");
    Console.WriteLine();
}

Console.WriteLine($"Total actions: {operation.ActionsTotal}");
Console.WriteLine($"  Succeeded actions: {operation.ActionsSucceeded}");
Console.WriteLine($"  Failed actions: {operation.ActionsFailed}");
Console.WriteLine($"  In progress actions: {operation.ActionsInProgress}");
Console.WriteLine();

await foreach (AnalyzeActionsResult documentsInPage in operation.Value)
{
    IReadOnlyCollection<ExtractKeyPhrasesActionResult> keyPhrasesResults = documentsInPage.ExtractKeyPhrasesResults;
    IReadOnlyCollection<RecognizeEntitiesActionResult> entitiesResults = documentsInPage.RecognizeEntitiesResults;

    Console.WriteLine("Recognized Entities");
    int docNumber = 1;
    foreach (RecognizeEntitiesActionResult entitiesActionResults in entitiesResults)
    {
        Console.WriteLine($" Action name: {entitiesActionResults.ActionName}");
        Console.WriteLine();
        foreach (RecognizeEntitiesResult documentResult in entitiesActionResults.DocumentsResults)
        {
            Console.WriteLine($" Document #{docNumber++}");
            Console.WriteLine($"  Recognized {documentResult.Entities.Count} entities:");

            foreach (CategorizedEntity entity in documentResult.Entities)
            {
                Console.WriteLine();
```

```
                    Console.WriteLine($"    Entity: {entity.Text}");
                    Console.WriteLine($"    Category: {entity.Category}");
                    Console.WriteLine($"    Offset: {entity.Offset}");
                    Console.WriteLine($"    Length: {entity.Length}");
                    Console.WriteLine($"    ConfidenceScore: {entity.ConfidenceScore}");
                    Console.WriteLine($"    SubCategory: {entity.SubCategory}");
                }
                Console.WriteLine();
            }
        }

        Console.WriteLine("Extracted Key Phrases");
        docNumber = 1;
        foreach (ExtractKeyPhrasesActionResult keyPhrasesActionResult in keyPhrasesResults)
        {
            Console.WriteLine($" Action name: {keyPhrasesActionResult.ActionName}");
            Console.WriteLine();
            foreach (ExtractKeyPhrasesResult documentResults in keyPhrasesActionResult.DocumentsResults)
            {
                Console.WriteLine($" Document #{docNumber++}");
                Console.WriteLine($"  Recognized the following {documentResults.KeyPhrases.Count} Keyphrases:");

                foreach (string keyphrase in documentResults.KeyPhrases)
                {
                    Console.WriteLine($"    {keyphrase}");
                }
                Console.WriteLine();
            }
        }
    }
}
```

Troubleshooting

General

When you interact with the Cognitive Services for Language using the .NET Text Analytics SDK, errors returned by the Language service correspond to the sa

For example, if you submit a batch of text document inputs containing duplicate document ids, a `400` error is returned, indicating "Bad Request".

```
try
{
    DetectedLanguage result = client.DetectLanguage(document);
}
catch (RequestFailedException e)
{
    Console.WriteLine(e.ToString());
}
```

You will notice that additional information is logged, like the client request ID of the operation.

```
Message:
    Azure.RequestFailedException:
    Status: 400 (Bad Request)

Content:
    {"error":{"code":"InvalidRequest","innerError":{"code":"InvalidDocument","message":"Request contains duplicated Ids. Make sure e

Headers:
    Transfer-Encoding: chunked
    x-aml-ta-request-id: 146ca04a-af54-43d4-9872-01a004bee5f8
    X-Content-Type-Options: nosniff
    x-envoy-upstream-service-time: 6
    apim-request-id: c650acda-2b59-4ff7-b96a-e316442ea01b
    Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
    Date: Wed, 18 Dec 2019 16:24:52 GMT
    Content-Type: application/json; charset=utf-8
```

Setting up console logging

The simplest way to see the logs is to enable the console logging. To create an Azure SDK log listener that outputs messages to console use AzureEventSou

```
// Setup a listener to monitor logged events.
using AzureEventSourceListener listener = AzureEventSourceListener.CreateConsoleLogger();
```

To learn more about other logging mechanisms see **here (https://github.com/Azure/azure-sdk-for-net/blob/main/sdk/core/Azure.Core/samples/Diagnos**

Next steps

Samples showing how to use this client library are available in this GitHub repository. Samples are provided for each main functional area, and for each area mode.

- **Detect Language (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample1_DetectLangu**
- **Analyze Sentiment (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample2_AnalyzeSer**
- **Extract Key Phrases (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample3_ExtractKey**
- **Recognize Named Entities (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample4_Rec**
- **Recognize PII Entities (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample5_Recogni**
- **Recognize Linked Entities (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample6_Rec**
- **Recognize Healthcare Entities (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample7_**
- **Custom Named Entities Recognition (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sar**
- **Custom Single Label Classification (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Samp**
- **Custom Multi Label Classification (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Samp**
- **Extractive Summarization (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample11_Extr**
- **Abstractive Summarization (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample12_A**

Advanced samples

- **Understand how to work with long-running operations (https://github.com/Azure/azure-sdk-for-net/blob/main/sdk/textanalytics/Azure.AI.TextAr**
- **Running multiple actions in one or more documents (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnaly**
- **Analyze Sentiment with Opinion Mining (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples**
- **Mock a client for testing (https://github.com/Azure/azure-sdk-for-net/tree/main/sdk/textanalytics/Azure.AI.TextAnalytics/samples/Sample_MockC**

Contributing

See the **CONTRIBUTING.md (https://github.com/Azure/azure-sdk-for-net/blob/main/CONTRIBUTING.md)** for details on building, testing, and contributin

This project welcomes contributions and suggestions. Most contributions require you to agree to a Contributor License Agreement (CLA) declaring that you **(https://cla.microsoft.com/)**.

When you submit a pull request, a CLA-bot will automatically determine whether you need to provide a CLA and decorate the PR appropriately (e.g., label, c using our CLA.

This project has adopted the **Microsoft Open Source Code of Conduct (https://opensource.microsoft.com/codeofconduct/)**. For more information see the **opencode@microsoft.com ()** with any additional questions or comments.

Downloads

Full stats → (/stats/packages/Azure.AI.TextAnalytics?groupby=Version)

Total **5.6M**

Current version **2.7M**

Per day average **2.7K**

About

Last updated 6/20/2023

Project website (https://github.com/Azure/azure-sdk-for-net/blob/Azure.AI.TextAnalytics_5.3.0/sdk/textanalytics/Azure.AI.TextAnalytics/README.md)

Source repository (https://github.com/Azure/azure-sdk-for-net)

MIT license (https://licenses.nuget.org/MIT)

Download package (https://www.nuget.org/api/v2/package/Azure.AI.TextAnalytics/5.3.0) (421.89 KB)

Open in NuGet Package Explorer (https://nuget.info/packages/Azure.AI.TextAnalytics/5.3.0)

Open in NuGet Trends (https://nugettrends.com/packages?ids=Azure.AI.TextAnalytics)

Report package (/packages/Azure.AI.TextAnalytics/5.3.0/ReportAbuse)

Owners

Contact owners → (/packages/Azure.AI.TextAnalytics/5.3.0/ContactOwners)

(/profiles/Microsoft) Microsoft (/profiles/Microsoft)

(/profiles/azure-sdk) azure-sdk (/profiles/azure-sdk)

Microsoft (/packages?q=Tags%3A%22Microsoft%22) Azure (/packages?q=Tags%3A%22Azure%22) Text (/packages?q=Tags%3A%22Text%22) Ar

(https://www.facebook.com/sharer/sharer.php?u=https://nuget.org/packages/Azure.AI.TextAnalytics/&t=Check+out+Azure.AI.TextAnalytics+on+%23NuGet!) (https://x.com/intent/post?url=https://nuget.org/packages/Azure.AI.

Contact (/policies/Contact)

Got questions about NuGet or the NuGet Gallery?

Status (https://status.nuget.org/)

Find out the service status of NuGet.org and its related services.

FAQ (Frequently Asked Questions) (https://aka.ms/nuget-faq)

Read the Frequently Asked Questions about NuGet and see if your question made the list.